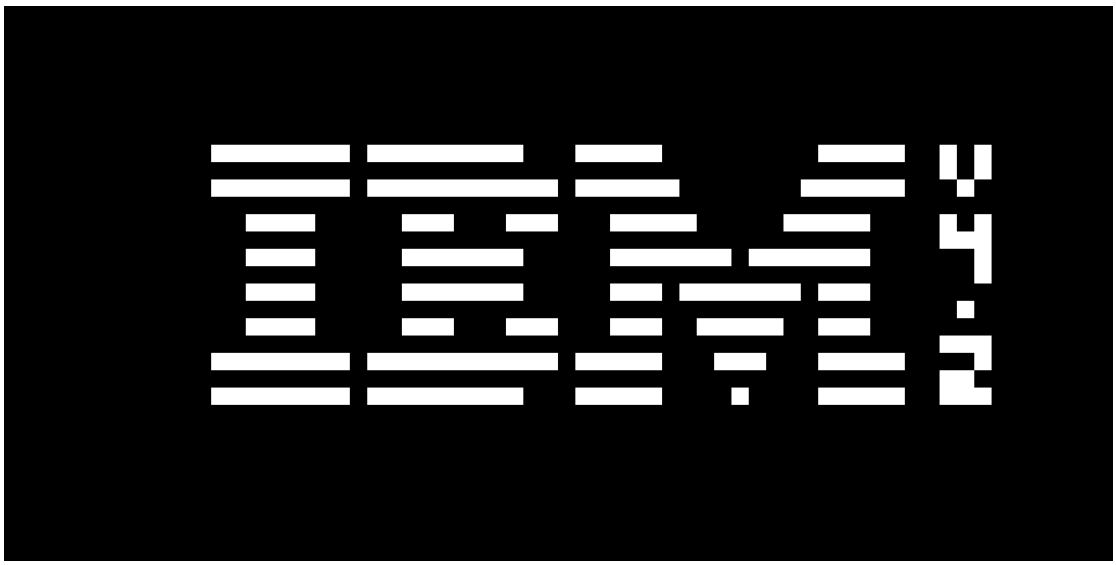
A very helpful guide to Chip8 I would recommend to help with everything:
https://tobiasvl.github.io/blog/write-a-chip-8-emulator/

This directory contains an unfinished Chip8 emulator. The `programs` directory contains Chip8 programs, from which the emulator tries to load IBM_Logo.ch8. Since the necessary instructions aren't implemented yet, it doesn't work, so it will just give you a black screen. Once you do implement them the emulator should be able to run the program and the window should look like this:



Most of the action is in the `src` directory.

- `src/cpu.c` contains everything you need to do in this task. The cpu_execute function is unfinished, and it has TODO comments for the instructions you need to implement yourself.

  These are the instructions you need to implement to get the :

`1NNN`	Jump to instruction
`6XNN`	Set register `VX` to `NN`
`7XNN`	Add `NN` to `VX` (without changing the carry bit)
`ANNN`	Set index register to `NNN`

  The instruction list at the end of this PDF is a good summary of all of them, and websites such as (https://tobiasvl.github.io/blog/write-a-chip-8-emulator/) go into much more detail when you need it.

- `src/main.c` is where the program starts. There is code for opening the window, displaying the pixels, loading Chip8 programs and setting up the emulator.
- `src/system.c` is where the RAM is set up, including copying font data and the program itself into it.

This repo uses the Raylib library for the window and graphics. It contains a couple sample programs from (https://github.com/Timendus/chip8-test-suite/), and you can find more from there too.

Machinecode	Octo Instruction	Comments
00E0	`clear`	
00EE	`return`	Exit a subroutine
1NNN	`jump NNN`	
2NNN	`NNN`	Call a subroutine
3XNN	`if vx != NN then`	
4XNN	`if vx == NN then`	
5XY0	`if vx != vy then`	
6XNN	`vx := NN`	
7XNN	`vx += NN`	
8XY0	`vx := vy`	
8XY1	`vx |= vy`	Bitwise OR
8XY2	`vx &= vy`	Bitwise AND
8XY3	`vx ^= vy`	Bitwise XOR
8XY4	`vx += vy`	vf = 1 on carry
8XY5	`vx -= vy`	vf = 0 on borrow
8XY6	`vx >>= vy`	vf = old least significant bit
8XY7	`vx =- vy`	vf = 0 on borrow
8XYE	`vx <<= vy`	vf = old most significant bit
9XY0	`if vx == vy then`	
ANNN	`i := NNN`	
BNNN	`jump0 NNN`	Jump to address NNN + v0
CXNN	`vx := random NN`	Random number 0-255 AND NN
DXYN	`sprite vx vy N`	vf = 1 on collision
EX9E	`if vx -key then`	Is a key not pressed?
EXA1	`if vx key then`	Is a key pressed?
FX07	`vx := delay`	
FX0A	`vx := key`	Wait for a keypress
FX15	`delay := vx`	
FX18	`buzzer := vx`	
FX1E	`i += vx`	
FX29	`i := hex vx`	Set i to a hex character
FX33	`bcd vx`	Decode vx into binary-coded decimal
FX55	`save vx`	Save v0-vx to i through (i+x)
FX65	`load vx`	Load v0-vx from i through (i+x)

The above instructions are sourced from (https://johnearnest.github.io/Octo/docs/chip8ref.pdf)